

JECINTA WANGUI

Full Stack Developer

Angular · TypeScript · Python / Django · REST APIs · PostgreSQL / SQL · Automated Testing
Nyeri, Kenya | 0111 969 356 | jecintawangui2001@gmail.com
github.com/Jess2001 | linkedin.com/in/jecintawangui | jesswangui.netlify.app

PROFESSIONAL SUMMARY

Full Stack Developer with approximately 3 years of professional software development experience across frontend and backend. Has shipped Angular and JavaScript/TypeScript interfaces and Python/Django REST APIs on production multi-tenant healthcare and SACCO banking platforms, and designed relational database schemas in PostgreSQL using transactions, constraints and concurrency-safe workflows. Strong in frontend/backend integration, SQL, query optimisation, automated testing (pytest, GitHub Actions CI), Git-based code review, and troubleshooting production issues with Sentry and logs. BSc in Computer Science. Analytical problem solver with strong communication, interpersonal skills and attention to detail; works independently and in a team. Based in Nyeri.

TECHNICAL SKILLS

Languages: Python, JavaScript, TypeScript, SQL

Frontend: Angular, JavaScript, TypeScript, RxJS, Angular Material, Chart.js, HTML5/CSS3, frontend/backend integration, REST API integration, form validation

Backend: Django, Django REST Framework, Django ORM, REST API design, JWT authentication, RBAC, service-layer architecture, Swagger/OpenAPI

Databases: PostgreSQL, SQL, MongoDB, MongoEngine, relational database schemas, schema design, query optimisation, indexing, transactions, data integrity

Testing & Quality: pytest, pytest-django, unit testing, integration testing, API testing, concurrency testing, regression testing, Postman, GitHub Actions CI, code review

Engineering: Git, GitHub, Docker, Linux, Sentry, Amazon S3 (pre-signed URLs), debugging, troubleshooting, root-cause analysis, system architecture, system design, refactoring, performance optimisation, data structures and algorithms

Professional: Communication, interpersonal skills, analytical thinking, problem solving, attention to detail, working independently and in teams

PROFESSIONAL EXPERIENCE

Izola Life — Software Developer (Frontend & Full-Stack)

Jul 2024 – Sep 2026

Xaidi, a multi-tenant healthcare SaaS platform | Kiambu, Kenya

- Built the **Angular** admin dashboard for appointments, therapists and organisations, with Chart.js charts and Angular Material tables, using **RxJS** (debounced, de-duplicated refreshes and takeUntil cleanup) for asynchronous data flows, and wrote the **Django REST** backend behind each feature.
- Built reporting dashboards with multi-attribute filtering (year, revenue stream, subscription type, payment status), backed by DRF and MongoDB aggregation APIs.
- Built a bulk user-onboarding feature (CSV/Excel upload with Papa Parse) with **form validation** on the UI and API: per-row checks for missing fields and invalid or duplicate emails, a downloadable template and invalid-rows file, import of valid rows only, and admin-only permissions on both layers.
- Structured the Angular app with **lazy-loaded feature modules**, route guards (auth, role, organisation) driven by per-route role data, an **HTTP interceptor** that attaches Firebase ID tokens to API calls, and a custom **directive** that limits role assignment to the user's own level; cached user-list requests with shareReplay and TTL invalidation.
- Built and maintained Django REST APIs (MongoDB/MongoEngine) for patient, therapist, appointment and onboarding workflows; implemented **RBAC and tenant isolation** across organisation, therapist and patient data, validating authorisation and isolation edge cases during development.
- Debugged** inconsistent paid-appointment and revenue figures: traced them to two features using different date fields (attendance vs payment date), corrected the calculation to use payment date, and regression-tested across payment methods and month boundaries.
- Debugged** a bulk-invoicing defect where actions only covered the visible table page (5 rows): the backend now counts all unpaid, attended appointments, prompts the user to invoice the selection or all, and generates invoices from the full list.
- Built an **audit-logging** system capturing actors, affected records and field-level changes, and optimised the activity-log APIs with pagination, selective field loading and in-memory caching to eliminate N+1 query patterns.
- Optimised database queries and aggregation workflows (filtering, lookups, grouping, indexing), contributing to approximately **30% lower query latency**.
- Refactored complex API logic into reusable **service-layer** components, separating API, business logic and data layers in the **system architecture** to improve maintainability and testability.

- Implemented therapist document-management APIs on **cloud services** (Amazon S3 pre-signed URLs) for secure uploads and resource validation.
- **Troubleshoot issues** in production across API requests, application logic, database records and third-party services using **Sentry**, logs and structured error handling.
- Reviewed colleagues' changes for correctness, maintainability and coding standards before staging merges; agreed **API contracts** and requirements with frontend developers and business teams across Kenya and Germany; documented REST APIs with Swagger/OpenAPI.

E and M Technology House — Frontend Developer Intern

Aug 2023 – Jan 2024

Multi-tenant SACCO banking platform | Tatu City, Kenya

- Built **Angular** interfaces for member accounts, loan tracking, contributions and transaction management for multiple financial institutions.
- Turned SACCO operating requirements from stakeholders into working Angular modules, iterating on feedback.
- Integrated and tested REST APIs with **Postman**, validating account and transaction data and catching defects before frontend integration and release.
- Collaborated with backend engineers on API contracts and defect resolution.

Au Innovations — Software Developer Intern

May 2022 – Aug 2022

- Supported Dynamics 365 Business Central workflows; wrote process documentation that cut support overhead by approximately 30%.

SELECTED PROJECTS

MobiLend — Digital Lending Platform (in progress)

Django REST Framework · PostgreSQL · pytest · Docker

- Built a custom user model with JWT authentication, 6-role RBAC, OTP verification (hashed codes, attempt limits) and password-reset flows, with scoped rate limiting on auth endpoints; least-privilege serializers hide sensitive customer fields.
- Designed the relational database schemas, separating customer-domain data from authentication identity, with constraints to protect data integrity; implemented an atomic registration workflow that commits three related records in one transaction and rolls back on failure.
- Prevented duplicate customer numbers under concurrent requests using row locking and unique constraints; generated sequential application numbers (APP-2026-00001) atomically from a locked sequence table.
- Modelled term-based loan product pricing (LoanProduct → LoanProductTerm) with database CHECK and UNIQUE constraints for amount ranges, rate bounds and one price per term, so invalid data is rejected at the database level; money is stored as Decimal and foreign keys use PROTECT.
- Built the loan-products listing endpoint with verified-user permissions, nested serializers, pagination and a filtered Prefetch, with a test asserting a constant 3 queries however many products exist.
- Wrote a pytest/pytest-django suite covering authentication, authorisation across all roles, OTP verification and rollback behaviour; added lending-domain tests for database constraints (savepoints, constraint-name assertions), parametrised boundary cases and 401/403 permission cases.
- Containerised the app with Docker and PostgreSQL; documented the API with OpenAPI (drf-spectacular).

Clinic Booking API

Django REST Framework · PostgreSQL · pytest · GitHub Actions

- Engineered a concurrency-safe scheduling API that prevents double-booking for near-simultaneous requests (~200ms apart), using select_for_update(), atomic transactions and a composite SQL unique constraint.
- Implemented JWT authentication that derives the user from the token, not client-supplied patient identifiers, protecting patient data.
- Wrote a **pytest-django** suite, including a ThreadPoolExecutor concurrency test, and configured **GitHub Actions CI** to block merges to main on any failing test.

EDUCATION

Bachelor of Science in Computer Science — Kabarak University, Nakuru, Kenya

2019 – 2023

Second Class Honours, Upper Division. Coursework: Data Structures, Database Management Systems, Software Architecture.